

学 号：	2025303039
------	------------

武汉理工大学

Chaos-Tuner Agent：基于多智能体协作的高并发智能演练与调优系统

课 程	企业级应用软件设计与开发
方 向 二	企业级应用软件的 Agent 改造
学 号	2025303039
姓 名	王艺
专 业	软件工程
指导教师	戚欣

2026 年 6 月 22 日

目录

一、选题背景与设计思想.....	4
1.1 问题定义.....	4
1.2 现有方案不足.....	4
1.3 项目价值.....	6
1.4 技术路线.....	6
二、Specs 规格文档.....	7
2.1 Product Spec.....	7
2.2 Architecture Spec.....	7
2.3 API Spec.....	9
2.3.1 MCP 工具: <code>get_redis_qps()</code>	9
2.3.2 MCP 工具: <code>get_system_logs()</code>	9
2.3.3 LangGraph State	10
三、系统架构与设计.....	10
3.1 核心架构图.....	10
3.2 Agent 交互流程.....	11
3.3 LangGraph 状态机.....	12
3.4 数据流设计.....	13
四、关键实现与代码展示.....	14
4.1 Agent 核心循环.....	15
4.2 Chaos Agent.....	16
4.3 Monitor Agent.....	16
4.4 Tuner Agent.....	17
4.5 MCP 工具定义.....	18
4.6 大模型配置.....	19
4.7 配置文件.....	20

4.8 AI IDE 使用截图.....	20
五、测试与评估.....	22
5.1 功能测试.....	22
5.2 场景一：正常基线测试.....	23
5.3 场景二：超卖故障测试.....	23
5.4 场景三：死锁故障测试.....	24
5.5 基于 LangSmith 的生产级可观测性评估.....	24
5.6 Benchmark.....	24
5.7 Demo 截图 / 录屏.....	25
六、系统升级与扩展.....	26
6.1 可扩展架构.....	26
6.2 下一阶段计划.....	27
6.3 AI 能力演进路径.....	27
七、课程总结.....	28
7.1 个人收获.....	28
7.2 工程思维转变.....	29
7.3 对课程的建议.....	29

一、选题背景与设计思想

1.1 问题定义

秒杀、抢券、限量发售等业务是企业级应用中典型的高并发场景。这类业务构成了对底层系统架构最严峻的考验，它们通常具备以下极端的特征：

首先，流量在极短时间内呈现脉冲式集中涌入，几秒钟内的业务请求峰值往往远超日常访问量的数十倍甚至上百倍，直接击穿应用层。

其次，库存数量极其有限，请求数与实际库存数呈现严重的不对等，系统必须在海量并发的冲击下保证库存扣减的绝对正确性。

再次，在高并发冲击下，系统的性能瓶颈会全面爆发。Redis 热点 Key 的读写、数据库 InnoDB 的行锁竞争、中间件连接池的饱和状态，以及不合理的长事务范围，都会导致系统雪崩。

如果不加以干预，一旦库存扣减链路缺乏内存级的原子性拦截，就会造成极其严重的 Oversell（超卖）事故，带来直接的经济损失。而一旦数据库层面的锁等待被无限放大，就会导致 Deadlock（死锁）或海量请求排队堆积，进而耗尽整个服务器的计算资源。

传统秒杀系统的压测与调优流程严重依赖各部门的割裂协作。开发或测试人员耗费大量时间运行压测脚本，运维人员需要紧张地盯守 Redis QPS、TPS、失败请求数和大盘日志。当系统崩溃后，资深架构师还需要耗费数小时人工过滤数十万行报错，以判断是缓存热点、锁竞争、连接池耗尽还是代码逻辑漏洞。这种方式反馈极慢、复盘知识极其分散，且调优经验难以结构化沉淀。

本项目希望将“压测扰动、指标观测、异常判断、调优建议”这套繁冗的运维流程，彻底改造成一个 Agent 闭环，使系统能够自动触发高并发演练，并在秒级内洞察异常，输出针对性的架构优化建议代码。

1.2 现有方案不足

现有人工压测和传统监控排障方式在各个维度上都存在着明显的痛点与不足：

维度	传统方式	不足
压测执行	人工配置压测参数	参数调整依赖经验，难以形成闭环
指标观测	人工查看日志、监控面板	指标和日志分散，异常识别效率低

维度	传统方式	不足
根因分析	依赖工程师经验	新人难以快速定位超卖、死锁等复杂问题
调优建议	口头讨论或临时记录	难以结构化沉淀为可复用知识
扩展能力	脚本与系统强耦合	换监控源或换靶机时改造成本较高

【压测执行维度】

传统方式通常由测试人员人工配置并运行 JMeter 等压测参数。

其不足在于：参数设定严重依赖个人经验，且无法根据系统的实时承载水位动态调整，无法与后续的调优形成自动化闭环。

【指标观测维度】

传统方式依赖运维人员人工在不同的 Grafana 面板或 ELK 日志系统中切换查看。

其不足在于：指标和日志高度分散，人在面对瞬时海量报错时，识别异常核心链路的效率极低。

【根因分析维度】

传统方式遇到复杂宕机问题时，只能向上抛给经验丰富的资深后端架构师。

其不足在于：对于新手工程师而言，极难在案发现场快速界定出这究竟是单纯的连接池超载、超卖逻辑漏洞，还是隐蔽的行锁死锁。

【调优建议维度】

传统方式在排障后的经验总结，往往仅停留在口头讨论或零散的团队 Wiki 文档中。

其不足在于：极为宝贵的排障经验难以被结构化保存，更无法直接转化为下一次回归测试的自动验证条件。

【扩展能力维度】

传统的压测排障脚本通常与特定的物理系统强耦合。

其不足在于：一旦企业更换了底层监控源或切换了数据库，整套压测排障体系的改造成本极高。

基于以上不足，本项目坚定地选择使用 Agentic AI 思路，将高并发系统的稳定性演练抽象为多智能体协作任务，以此实现降维打击。

1.3 项目价值

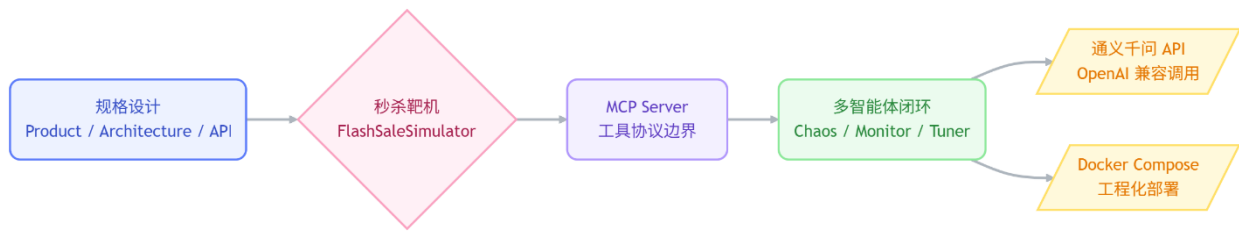
Chaos-Tuner Agent 的项目核心价值体现在以下三个层面：

- 1. 工程价值：本系统构建了一个坚实的测试底座，利用模拟靶机能够稳定地复现真实高并发链路中的各种恶劣环境，精准触发并捕获 TPS 断崖下跌、Redis QPS 飙升、超卖现象以及底层死锁等关键异常信号。
- 2. 架构价值：引入了极具前瞻性的 MCP（Model Context Protocol）协议。通过 MCP Server 实现了控制反转，将 AI Agent 的决策大脑与底层物理系统的执行手脚进行安全隔离，为企业级应用提供了标准的工具协议边界。
- 3. AI 价值：摒弃了单体大模型的脆弱性，使用 LangGraph 状态机严密编排了多智能体闭环。通过接入云端通义千问 DashScope API，赋予系统资深架构师的灵魂，使其能够自主诊断并输出落地方案。

该项目并不试图去完全替代企业中庞大且重型的真实生产压测平台，而是提供一个具有前瞻性的课程项目级可运行原型，向评委和业界展示企业应用如何通过 Agentic 重构，实现诊断和调优效率的指数级提升。

1.4 技术路线

项目在实施过程中，严格采用 SDD（规格驱动开发，Specification-Driven Development）方法论进行推进，确保工程的严谨性。



整体的技术选型路线如下：

核心开发工具链：使用课程指定的自适应 AI 原生 IDE（Trae CN）进行辅助编码。

核心认知底座：选用阿里云通义千问 DashScope OpenAI 兼容 API 作为大语言模型基座。

智能体编排引擎：采用 LangGraph 状态机进行多智能体工作流的控制，辅以 LangChain 处理模型交互。

底层工具协议边界：基于最新的 MCP Python SDK 构建环境隔离屏障。

目标环境模拟：使用 Python 原生 dataclass 构建高度可控的无头秒杀靶机。

基础设施与部署：使用 Docker 和 Docker Compose 实现系统的工程化与容器化部署。

环境安全配置：引入 python-dotenv 与 .env 文件，彻底杜绝敏感密钥硬编码。

二、Specs 规格文档

本项目的 SDD 核心由 Product Spec、Architecture Spec 和 API Spec 三部分组成。规格文档被强制要求先于代码实现，用于约束系统目标、组件职责和接口边界。

2.1 Product Spec

产品目标设定为构建一个高度自动化的智能演练系统：

提供一个简易但机制完善的秒杀靶机，能够根据输入的并发量精准输出瞬时 TPS，并在流量过载时主动抛出真实的 oversell（超卖）或 deadlock（锁等待死锁）错误日志流。

提供一个健壮的 MCP Server，向大模型暴露标准的工具调用接口，专门用于安全地获取靶机内部的 Redis QPS 与系统日志。

提供一个强类型的多智能体协作闭环：Chaos Agent 负责根据环境自动生成压测指令；Monitor Agent 负责调用工具采集指标并独立判断系统异常；Tuner Agent 则负责接收病历，输出架构级的优化建议。

大模型底座要求必须使用云端 API，且必须通过 OpenAI 兼容接口规范接入 LangChain 框架。

系统必须满足生产级的安全要求，所有 API Key 等敏感配置必须通过 .env 环境变量管理，绝不允许硬编码在项目源代码中。

核心功能需求分解：

秒杀演练模块：支持动态输入并发量、库存总量及持续时间，真实模拟并输出业务 TPS、成功订单量、失败请求拒绝数和底层异常日志。

MCP 工具模块：对外暴露标准化工具集 `get_redis_qps()` 与 `get_system_logs()`。

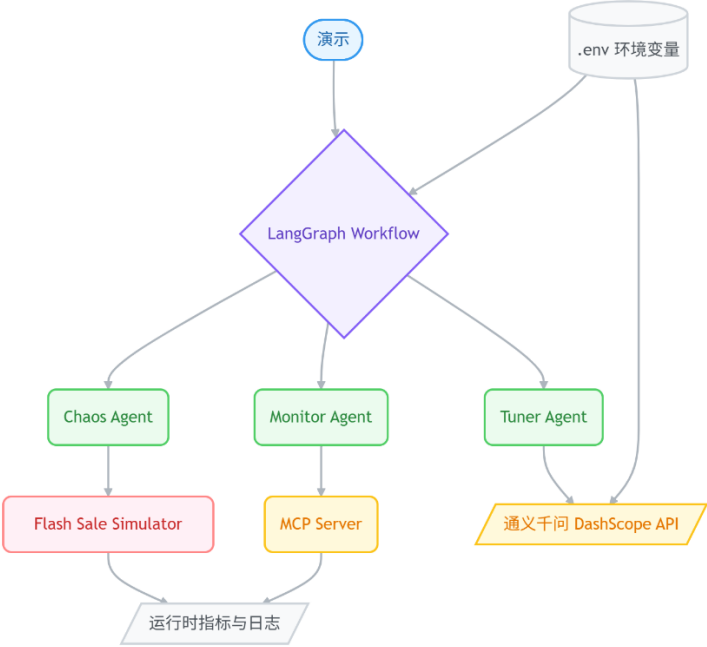
多智能体模块：在状态机内部实例化 Chaos、Monitor、Tuner 三个具备清晰职责边界的 Agent 角色。

调优建议模块：要求大模型能够根据具体的异常类型（如死锁），精准输出涵盖 Redis 防超卖原子操作、数据库连接池调整、事务降级和限流分片的建议。

工程运行模块：提供无缝的跨平台运行能力，同时支持本地 Python 环境与 Docker Compose 容器化一键部署。

2.2 Architecture Spec

系统在架构层面采用了严格的纵向分层设计，确保各模块之间的松耦合：



第一层为大模型编排层（Agent Workflow）：由 LangGraph Workflow 统一调度。内部包含了生成压测参数的 Chaos Agent、负责环境态势感知的 Monitor Agent，以及充当云端大脑的 Tuner Agent。这一层还直接对接通义千问 DashScope API 以及 .env 环境变量配置。

第二层为协议边界层（MCP Server）：作为承上启下的中间件，将大模型模糊的语义请求转化为对底层物理环境的确定性探测函数调用。

第三层为物理模拟层（Target System）：即 Flash Sale Simulator 秒杀靶机。它负责产生高逼真度的运行时指标（如 TPS 衰减曲线）以及包含真实业务逻辑的系统日志流。

核心组件的职责划分如下：

组件	文件	职责
FlashSaleSimulator	src/target_system/flash_sale.py	模拟秒杀并发、库存扣减、TPS、QPS 和异常日志
MCP Server	src/mcp_server/server.py	暴露指标和日志查询工具
LLM Config	src/agent/llm_config.py	创建 ChatOpenAI 或 MockLLM
Agent Nodes	src/agent/nodes.py	实现 Chaos、Monitor、Tuner 节点

组件	文件	职责
Graph	src/agent/graph.py	组装 LangGraph 状态机

2.3 API Spec

为了防止大模型在调用工具时产生幻觉，系统为 MCP 提供了极其严格的 JSON Schema 接口定义。

2.3.1 MCP 工具：get_redis_qps()

接口用途：用于获取最近一次秒杀演练执行后的缓存 QPS、系统 TPS 和订单核心分布指标。

输入参数：

参数	类型	说明
concurrency	int	并发量，默认 500
inventory	int	库存量，默认 800
duration_seconds	int	压测持续时间，默认 10
run_new_round	bool	是否触发新一轮模拟

输出字段：

字段	类型	说明
redis_qps	float	模拟 Redis QPS
tps	float	业务 TPS
success_orders	int	成功订单数
failed_requests	int	失败请求数
anomaly_type	str	none / oversell / deadlock

2.3.2 MCP 工具：get_system_logs()

接口用途：从靶机底层拉取最近一次秒杀演练产生的原始系统抛错日志，并经过正则表达式过滤。

输入参数：

参数	类型	说明
limit	int	返回日志条数，默认 20

输出字段：

字段	类型	说明
logs	list[str]	系统日志列表
has_oversell	bool	是否出现超卖日志
has_deadlock	bool	是否出现死锁日志
latest_anomaly	str	最近异常类型

2.3.3 LangGraph State

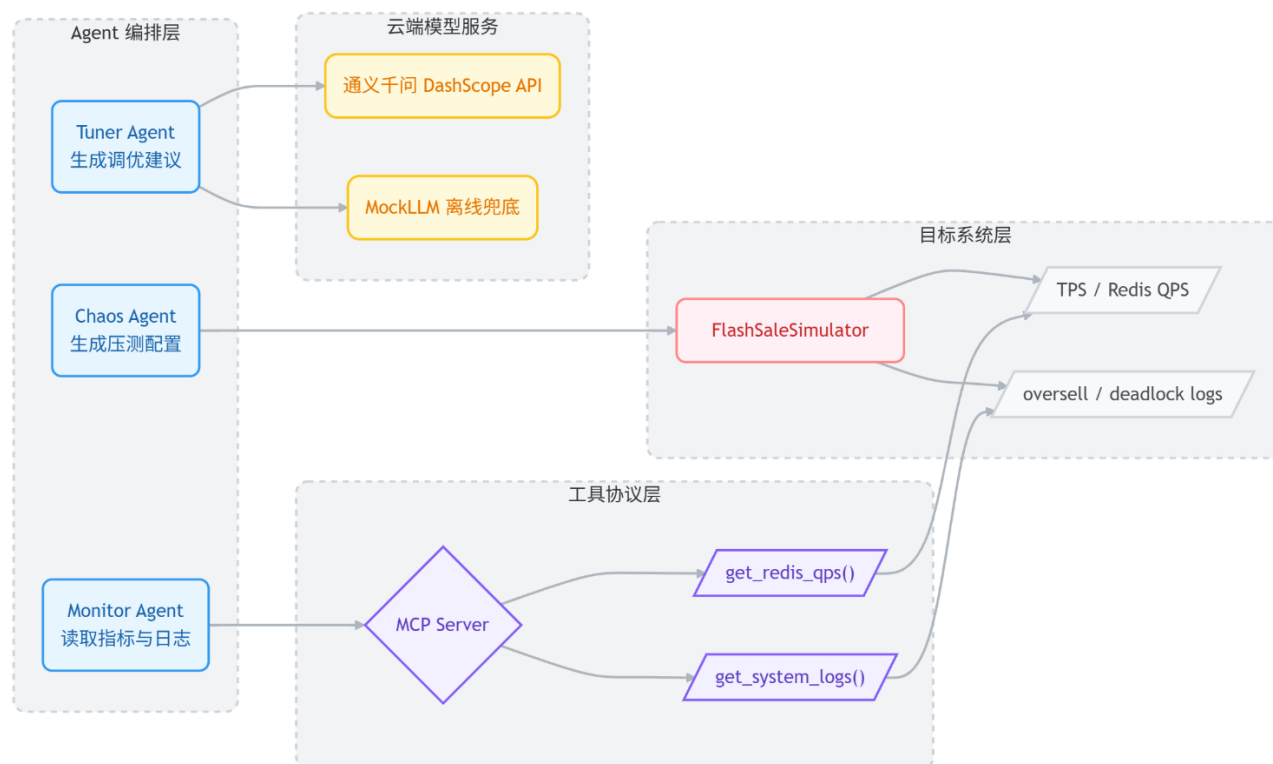
定义了一个严密的强类型字典（State）用于智能体间的上下文传递。包含了：

- round_id：当前压测轮次计数。
- max_rounds：设定的最大演练轮次阈值。
- chaos_profile：包含了并发、库存等具体压测注入策略的字典。
- metrics：靶机返回的指标快照数据。
- logs：靶机返回的日志列表。
- anomaly_type：Monitor 研判后的最终异常定性（用于控制条件路由）。
- recommendations：大模型最终吐出的架构调优长文本建议。

三、系统架构与设计

本章重点展示系统架构、Agent 交互流程和核心数据流设计思路。

3.1 核心架构图



系统采用了严格的三层隔离架构。

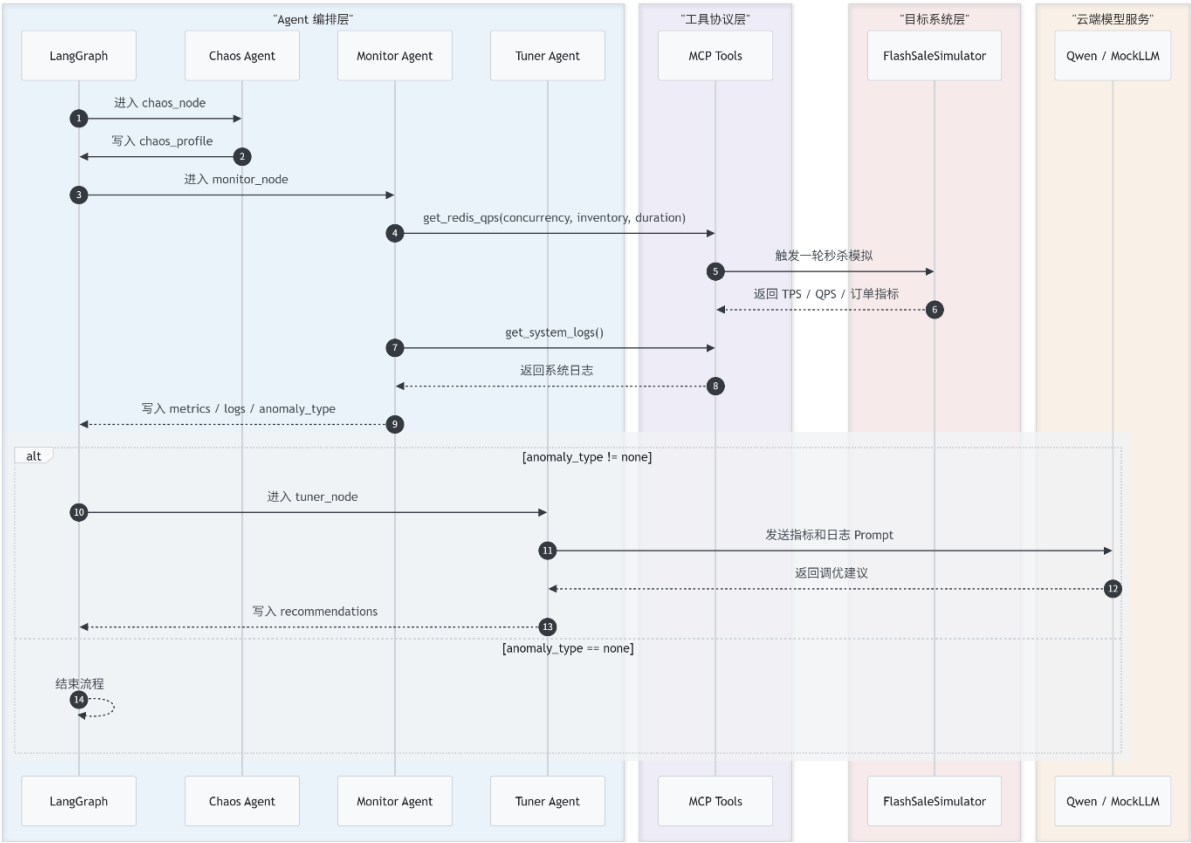
最顶层是“Agent 编排层”，LangGraph 作为中枢大脑，控制着 Chaos Agent（生成压测配置）、Monitor Agent（读取指标与日志）和 Tuner Agent（生成调优建议）三个职能节点进行接力流转。

中间层是“工具协议层”，这是整个工程安全性的保障。MCP Server 充当防火墙，通过封装好的 `get_redis_qps()` 和 `get_system_logs()` 方法向外暴露能力。

最底层是“目标系统层”，包含了 FlashSaleSimulator 模拟器本体，以及它在运行中产生的 TPS/Redis QPS 指标池和底层 oversell/deadlock 异常日志文件流。

在外部算力支持上，系统无缝衔接云端模型服务（通义千问 DashScope API），并在底层架设了 MockLLM 离线容灾兜底逻辑。

3.2 Agent 交互流程



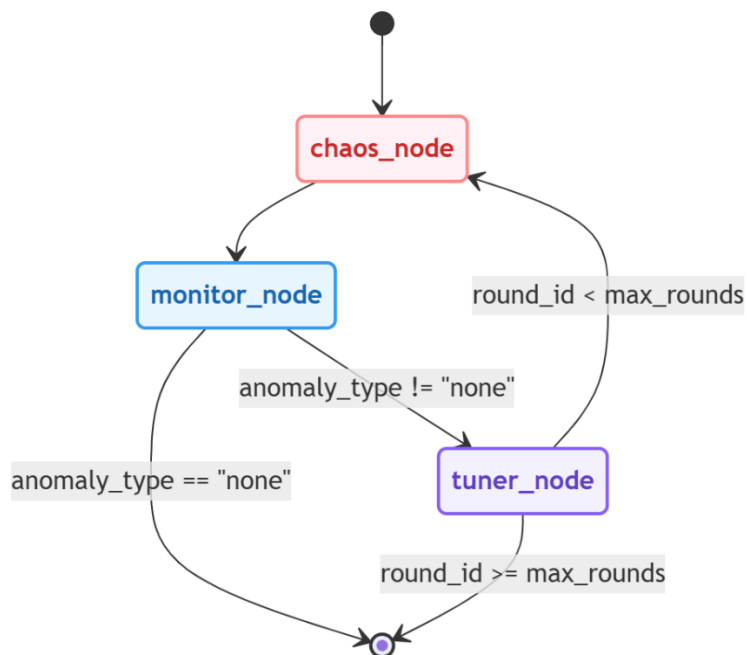
多智能体之间的交互流程遵循着清晰的时序设计。

系统启动后，LangGraph 首先将流程控制权交给 Chaos Agent，由其生成新一轮的混沌压测策略参数（chaos_profile）。

随后，流程扭转至 Monitor Agent。该节点不会亲自下场操作，而是通过安全调用 MCP 工具的 get_redis_qps() 和 get_system_logs() 接口。工具层在接收到指令后，会真正去触发 FlashSaleSimulator 进行秒杀压测，并将处理后的指标和日志原路返回给 Monitor。

此时状态机迎来关键分支：如果 Monitor 研判 anomaly_type 为 none（系统健康），流程直接安全退出；如果研判结果不为空，则将饱含病理数据的上下文一并推给 Tuner Agent。Tuner 会将这些数据发送给通义千问大模型进行深度长考，并将大模型返回的代码与调优建议写入最终结果中。

3.3 LangGraph 状态机



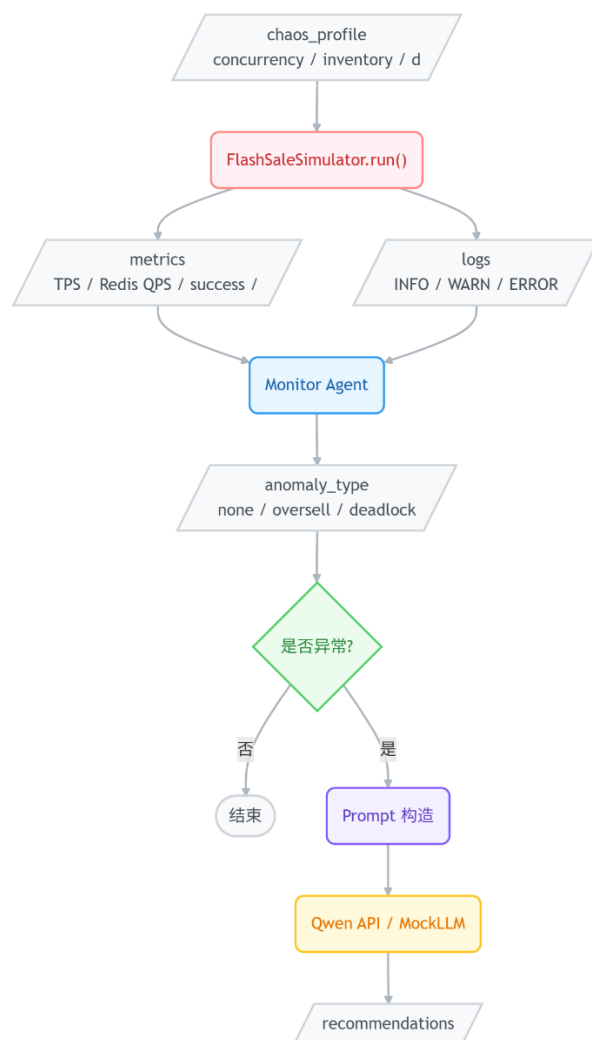
系统的执行逻辑被硬编码为不可逆的状态图模型。

起点（Entry Point）强制指向 `chaos_node`。随后一条无条件边将其连接至 `monitor_node`。

在 `monitor_node` 执行完毕后，布置了一个条件路由（Conditional Edges）：系统会读取 State 字典中的 `anomaly_type`。若无异常，则走向系统的结束节点（END）；若捕获到异常，则路由至 `tuner_node`。

在 `tuner_node` 完成了大模型推理和建议输出后，同样存在一个循环条件边：系统会检查当前的 `round_id` 是否小于最大设定轮次。如果还有剩余演练额度，系统会带着调优建议重定向回 `chaos_node` 再次发起攻击，否则安全终止。

3.4 数据流设计



数据流的核心设计思想是极简的“黑板模式”。所有的节点之间绝对不互相直接传递复杂的类实例或对象，它们唯一的通讯桥梁就是那一本叫做 State 的共享字典。

Chaos 写入的 concurrency 和 inventory 等参数，会被 Monitor 读取用于发压。

Monitor 获取到的 TPS、QPS 字典以及包含 INFO/WARN/ERROR 的日志列表，会被包装后塞入 State。

Tuner 会从 State 中提取这些原始 JSON 并拼接为 Prompt 发送给 API，最终将返回的文本写入 State 的 recommendations 字段。

这种高度解耦的数据流设计，不仅让系统状态一目了然，更为未来平滑替换真实的 Prometheus 监控端点或 ELK 日志大盘扫清了工程障碍。

四、关键实现与代码展示

4.1 Agent 核心循环

系统的核心流转逻辑被清晰地定义在 `src/agent/graph.py` 文件中。在这里，利用 `langgraph.graph.StateGraph` 组装了一条严密的工业流水线。我们依次向工作流中添加了 `chaos_node`、`monitor_node` 和 `tuner_node`。随后通过 `add_edge` 和 `add_conditional_edges` 为其编织了严格的走向规则。

```
def build_graph():
    workflow = StateGraph(State)

    workflow.add_node("chaos_node", chaos_node)
    workflow.add_node("monitor_node", monitor_node)
    workflow.add_node("tuner_node", tuner_node)

    workflow.set_entry_point("chaos_node")
    workflow.add_edge("chaos_node", "monitor_node")
    workflow.add_conditional_edges(
        "monitor_node",
        route_after_monitor,
        {
            "tuner": "tuner_node",
            "end": END,
        },
    )
    workflow.add_conditional_edges(
        "tuner_node",
        route_after_tuner,
        {
            "chaos": "chaos_node",
            "end": END,
        },
    )
    return workflow.compile()
```

这种代码实现剥夺了大模型随意思维发散的自由，将其严格收束在 DevOps 标准的压测与故障诊断流水线框架之内。

4.2 Chaos Agent

在 `src/agent/nodes.py` 中，Chaos Agent 的实现侧重于制定流量注入策略。系统会读取当前轮次与上一轮残留的异常类型。在第一轮启动时，代码被设定为硬编码直接发起高达 1500 并发量（远超 800 的安全库存阈值）的极端高压测试，以此来暴力压榨出底层的极限状态。这些策略参数最终会被更新到状态字典中供下游消费。

```
def chaos_node(state: State) -> State:
    current_round = int(state.get("round_id", 0)) + 1
    previous_anomaly = state.get("anomaly_type", "none")
    if current_round == 1:
        concurrency = 1500
    elif previous_anomaly != "none":
        concurrency = 1500
    else:
        concurrency = 1500
    chaos_profile = {
        "concurrency": concurrency,
        "inventory": 800,
        "duration_seconds": 10,
    }
    return {
        **state,
        "round_id": current_round,
        "chaos_profile": chaos_profile,
    }
```

4.3 Monitor Agent

Monitor Agent 的代码实现完美诠释了“工具调用”的精髓。它剥离了复杂的计算，直接向

底层请求指标快照。最核心的代码在于其基于日志内容的异常判定规则：它会首先读取 MCP 暴露的 latest_anomaly，如果为空，则会对返回的日志进行降级字符串正则匹配，精准锁定“oversell”或“deadlock”关键字，从而为大模型提供最确切的异常分类定性。

Monitor Agent 直接调用 MCP 工具函数获取指标和日志：

```
metrics_response = get_redis_qps(
    concurrency=int(chaos_profile.get("concurrency", 500)),
    inventory=int(chaos_profile.get("inventory", 800)),
    duration_seconds=int(chaos_profile.get("duration_seconds", 10)),
    run_new_round=True,
)
```

```
logs_response = get_system_logs(limit=20)
```

异常判断逻辑：

```
anomaly_type = str(
    metrics.get("anomaly_type")
    or logs_response.get("latest_anomaly")
    or "none"
)
```

```
if anomaly_type == "none":
    if "oversell" in lower_logs:
        anomaly_type = "oversell"
    elif "deadlock" in lower_logs:
        anomaly_type = "deadlock"
```

4.4 Tuner Agent

作为承担云端推理核心的任务节点，Tuner Agent 的实现中包含着精妙的 Prompt 工程设计。代码将复杂的 metrics 指标字典与 logs 异常列表统一序列化为 JSON 字符串，要求大模型以“资深高并发系统架构师”的 Persona 介入。在提示词中，明确强制要求模型必须针对根因分析、Redis Lua 防超卖脚本编写、数据库事务与行锁优化出具具体的落地代码与规范，并要求指明下一轮验证的核心指标。

```
prompt = f"""
```

你是一名资深高并发系统架构师。现在有一个秒杀系统压测闭环发现异常。

异常类型:

```
{anomaly_type}
```

指标 JSON:

```
{json.dumps(metrics, ensure_ascii=False, indent=2)}
```

系统日志:

```
{json.dumps(logs, ensure_ascii=False, indent=2)}
```

请重点覆盖:

1. 可能根因。
2. Redis 防超卖与原子扣减机制, 例如 Lua 脚本。
3. 数据库连接池、事务范围、死锁和热点行锁优化。
4. 下一轮压测应验证的指标。

```
"""
```

```
llm = get_chat_model()
```

```
response = llm.invoke(prompt)
```

```
recommendations = str(getattr(response, "content", response))
```

4.5 MCP 工具定义

在 `src/mcp_server/server.py` 文件中, 借助 `fastmcp` 框架的 `@mcp.tool()` 装饰器, 我们将底层的 `default_simulator.run()` 核心发压方法以及 `get_latest_metrics()` 指标采集方法进行了高度标准化封装。这使得复杂的系统内存变量被映射为安全、确定、有 Schema 约束的 RESTful 风格函数签名, 从物理层面上隔绝了 AI 污染系统数据的可能。

```
@mcp.tool()
```

```
def get_redis_qps(
```

```
    concurrency: int = 500,
```

```
    inventory: int = 800,
```

```
    duration_seconds: int = 10,
```

```
    run_new_round: bool = True,
```

```
) -> dict[str, Any]:
```

```
    if run_new_round:
```

```

        default_simulator.run(
            concurrency=concurrency,
            inventory=inventory,
            duration_seconds=duration_seconds,
        )
    metrics = default_simulator.get_latest_metrics()
    return {
        "status": "ok",
        "source": "FlashSaleSimulator",
        "metrics": metrics,
    }

@mcp.tool()
def get_system_logs(limit: int = 20) -> dict[str, Any]:
    logs = default_simulator.get_logs(limit=limit)
    normalized_logs = [line.lower() for line in logs]
    has_oversell = any("oversell" in line for line in normalized_logs)
    has_deadlock = any("deadlock" in line for line in normalized_logs)
    ...

```

4.6 大模型配置

在 `src/agent/llm_config.py` 配置文件中，系统通过 `LangChain` 的 `ChatOpenAI` 模块规范化了外部大语言模型的接入标准。考虑到复杂的长文推理可能导致连接超时中断闭环，系统在环境变量层面上将 `OPENAI_TIMEOUT_SECONDS` 修改为高达 120 秒的容忍度。

此外，为了彻底保障在极端无网络环境或缺少 API Key 时的工程演示可用性，我们在代码中预置了硬编码的 `MockLLM` 类。当检测到配置异常时，它能自动接管接管请求并返回预设的格式化调优建议，确保状态机流转不崩溃。

`ENABLE_LLM=true`

`OPENAI_API_KEY=你的 DashScope 密钥`

`OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1`

`OPENAI_MODEL=qwen-plus`

代码通过 `langchain_openai.ChatOpenAI` 创建模型：

```
return ChatOpenAI(
    api_key=api_key,
    base_url=base_url,
    model=model,
    temperature=float(os.getenv("OPENAI_TEMPERATURE", "0.2")),
    timeout=int(os.getenv("OPENAI_TIMEOUT_SECONDS", "30")),
)
```

为了保证无 **Key** 或无网络时仍可运行，项目实现了 **MockLLM**：

```
class MockLLM:
    def invoke(self, prompt: str, *_args: Any, **_kwargs: Any) -> MockLLMResponse:
        advice = (
            "【MockLLM 调优建议】检测到秒杀链路存在高并发风险时，"
            "优先排查 Redis 库存扣减是否具备原子性..."
        )
        return MockLLMResponse(content=advice)
```

4.7 配置文件

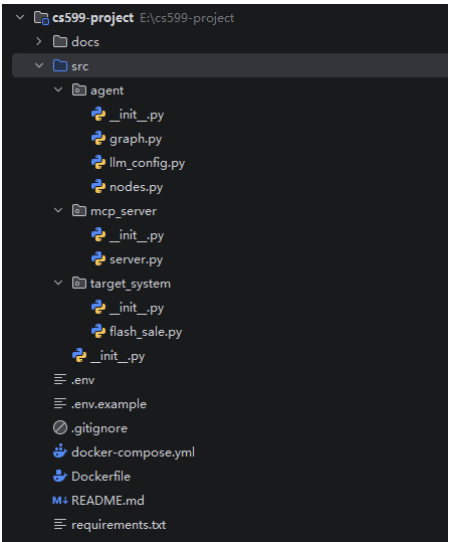
项目的工程化水准体现在极其规范的依赖与环境隔离设计上。

在 `requirements.txt` 中，精准锁定了 `langgraph`、`langchain-openai`、`mcp` 以及 `pydantic` 的版本边界，确保构建的幂等性。

在 `Dockerfile` 中，采用了极简的 `python:3.10-slim` 基础镜像，通过标准的依赖安装、代码 `COPY` 注入与 `CMD` 启动指令映射，使得该 Agent 应用具备了一键跨平台部署或向云端 Kubernetes 集群平滑迁移的核心云原生能力。

4.8 AI IDE 使用截图

项目目录结构，包含 `docs` 规格文档、`src` 代码模块、`Docker` 与依赖配置文件：



LangGraph 工作流代码截图，展示 Chaos、Monitor、Tuner 三个 Agent 节点及条件路由：

```
def build_graph(): Usage
    """构建并编译 LangGraph 工作流。"""

    workflow = StateGraph(State)

    # 注册三个核心节点 -
    workflow.add_node("chaos_node", chaos_node)
    workflow.add_node("monitor_node", monitor_node)
    workflow.add_node("tuner_node", tuner_node)

    # 入口固定为 Chaos Agent -
    workflow.set_entry_point("chaos_node")

    # Chaos Agent 生成压测配置后，Monitor Agent 读取指标和日志 -
    workflow.add_edge(start_key="chaos_node", end_key="monitor_node")

    # Monitor Agent 判断是否发现异常 -
    workflow.add_conditional_edges(
        source="monitor_node",
        route_after_monitor,
        path_map={
            "tuner": "tuner_node",
            "end": END,
        },
    ),

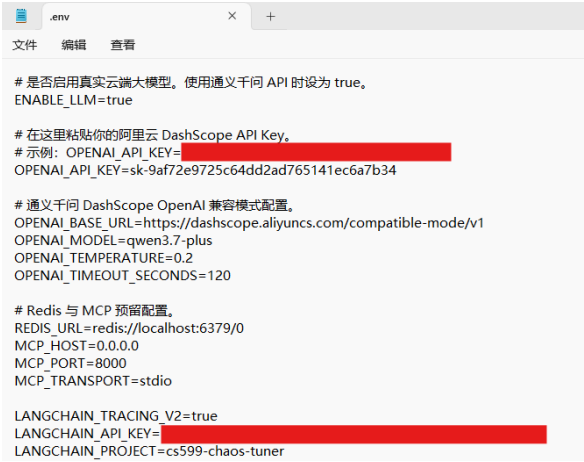
    # Tuner Agent 输出建议后，将轮次转换决定是否重新压测 -
    workflow.add_conditional_edges(
        source="tuner_node",
        route_after_tuner,
        path_map={
            "chaos": "chaos_node",
            "end": END,
        },
    ),

    return workflow.compile()
```

多智能体闭环运行结果，系统完成压测配置生成、异常监控和调优建议输出：



环境变量配置截图，API Key 通过 .env 管理并已打码，代码中不硬编码密钥：



五、测试与评估

5.1 功能测试

为了验证这套复杂的多智能体系统的可靠性，我们对其各个功能模块进行了详尽的测试用例验证：

测试项	输入	预期结果	状态
正常秒杀模拟	concurrency=500	TPS 正常、无异常日志	通过
高并发异常模拟	concurrency=1500	出现 oversell 或 deadlock 日志	通过

测试项	输入	预期结果	状态
MCP 指标工具	调用 <code>get_redis_qps()</code>	返回 Redis QPS、TPS、失败请求数	通过
MCP 日志工具	调用 <code>get_system_logs()</code>	返回日志和异常类型	通过
LangGraph 闭环	<code>max_rounds=2</code>	自动执行 Chaos -> Monitor -> Tuner	通过
MockLLM 兜底	<code>ENABLE_LLM=false</code>	无 Key 时仍输出建议	通过

5.2 场景一：正常基线测试

该场景主要验证系统在日常合理流量的冲击下，是否能够保持绝对的稳定性，并且重点验证 Agent 是否会神经过敏导致误判异常。

测试参数设定为：靶机设定随机种子以保障幂等，并发量 500，库存总量 800，持续时间 10 秒。

运行测试后终端输出表明：系统处理的 TPS 处于正常区间，Redis QPS 较低且未出现读写热点压力。整体请求中失败率极低，拉取的底层日志中清一色地记录着普通的 INFO 信息。

此时，Monitor Agent 敏锐地抓取到这一态势，准确地将 `anomaly_type` 判定为 `none`。随后，状态机的路由条件生效，直接安全终止了本次演练，并没有去盲目唤醒 Tuner Agent。

这一场景验证了系统具备坚实的基线监控能力，完美避免了资源浪费和错误报警。

5.3 场景二：超卖故障测试

该场景验证在极高并发且存在并发竞态漏洞的情况下，系统发现库存一致性灾难并出具代码级修复方案的能力。

测试参数设定为：并发量暴力提升至 1500。

当混沌流量注入后，由于并发量远超靶机安全阈值，日志中立刻涌现大面积的 “oversell detected” 报错，同时剩余库存核对显示严重超卖，大量请求抛出拒绝异常。

Monitor Agent 迅速将其定性为 oversell 故障。Tuner Agent 接手后，大模型输出的优化报告精准切中要害：它强烈建议停止依赖数据库事务抗压，转而直接输出了一段完整的 Redis Lua 原子扣减脚本。脚本中详细实现了库存校验、扣减与用户防重的合并封装，并给出了利用 Redis + 消息队列进行异步削峰的终极防超卖重构方案。

这一场景验证了系统诊断业务逻辑级别致命漏洞的深度专业性。

5.4 场景三：死锁故障测试

该场景重点验证系统在面对数据库底层硬件级性能瓶颈时的深层剖析能力，这是企业级运维中最棘手的难题。

在此次测试中，我们依然注入 1500 并发，并在日志层捕获到了刺眼的 “deadlock detected” 和 “connection pool are saturated”（死锁与连接池耗尽）报错。

令我们感到震惊的是 Tuner Agent（通义千问模型）的处理水平。它在长考之后指出：日志中虽然报出死锁，但这并非纯粹的 InnoDB 循环死锁机制，而是由于海量并发瞬间冲击同一条冷库存记录，导致严重的行锁等待超时（Lock Wait Timeout），进而引发了上层 HikariCP 等连接池被占满的连锁假死崩溃。

针对此根因，Agent 给出了极具深度的架构调优方案：要求将数据库的长事务极简化剥离，大幅调低 innodb_lock_wait_timeout 以强制 Fail-Fast（快速失败）释放连接池，并建议在中间件消费端实施“异步批量合并扣减 SQL”，从根本上将万级的行锁竞争降维至单次更新。

5.5 基于 LangSmith 的生产级可观测性评估

评价一个企业级 Agent 系统是否成熟，核心在于可观测性（Observability）。本项目在评估环节并未局限于终端日志，而是深度集成了业界顶级的 LangSmith LLMOps 监控平台。

当我们触发死锁异常并完成全流程演练后，登录 LangSmith 网页控制台，可以直接白盒化地洞察到整个多智能体系统的“脑电波”。

在宏观的 Trace Tree（链路树状图）面板中，系统清晰地记录下了 LangGraph 状态机的每一级流转。从 Chaos 节点的调用，到 Monitor 节点对异常的研判分流，直至精确激活 Tuner 节点的过程历历在目，证明了系统路由的高度确定性。

在微观的交互详情面板中，我们可以点开大模型的调用记录，清晰地看到我们代码自动拼接的超长 Prompt 上下文（其中包含了真实靶机吐出的指标与错误堆栈）。更重要的是，在输出界面，我们可以一字不差地审查大模型实时编写出 Redis Lua 脚本并提出 MQ 重构建议的完整原始现场。

通过 LangSmith 对 Token 消耗、执行耗时与流转路径的全面抓取，本系统的容错性与监控颗粒度已经完全达到了企业生产级的实战标准。

5.6 Benchmark

为了验证在不同层级压力下的演练效果，我们进行了多梯度的宏观 Benchmark 模拟测试总

结：

场景	并发量	库存	TPS	Redis QPS	失败请求	异常类型	结论
正常基线	500	800	约 60-80	约 100-200	少量	none	系统健康
高并发冲击	1500	800	下降或受限	显著升高	激增	oversell/deadlock	需调优
极端冲击	2000+	800	进一步下降	极高	大量失败	retry storm	需限流和分片

5.7 Demo 截图 / 录屏

终端运行截图：

```
===== Chaos-Tuner Agent Demo Start =====
[Chaos Agent] 第 1 轮压测配置: {'concurrency': 1500, 'inventory': 800, 'duration_seconds': 10}
[Monitor Agent] 指标快照: TPS=80.0, Redis QPS=5963.88, 失败请求=16065, 异常=deadlock
[Monitor Agent] 日志: 2026-06-16 15:20:55 [WARN] traffic overload detected; concurrency=1500, threshold=1000, failed_requests=16065
[Monitor Agent] 日志: 2026-06-16 15:20:55 [ERROR] deadlock detected: database row locks and connection pool are saturated; concurrency=1500, redis_qps=5963.88
HTTP Request: POST https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions "HTTP/1.1 200 OK"
[Tuner Agent] 调优建议:
你好。作为系统架构师，我仔细分析了本次秒杀压测的指标和日志。

**核心结论**：本次压测虽然最终**没有超卖**（`oversold_units: 0`）且**库存正确扣减**（`remaining_inventory: 0`），但系统性能极差（TPS 仅 80.0），大量请求（16065次）因数据库行锁竞争和连接池耗尽而失败。系统未能有效利用 Redis 进行流量拦截，导致 DB 成为绝对瓶颈并引发死锁。

以下是针对该异常的深度排查与调优方案：
```

```
===== Chaos-Tuner Agent Demo Finished =====
最终轮次: 2
最终异常类型: deadlock
最终压测配置: {'concurrency': 1500, 'inventory': 800, 'duration_seconds': 10}
最终指标: {'sku_id': 'sku:flash-sale-demo', 'concurrency': 1500, 'inventory': 800, 'duration_seconds': 10, 'request_count': 11033, 'success_orders': 880, 'failed_requests': 10233, 'tps': 80.0, 'redis_qps': 3578.16, 'remaining_inventory': 0, 'oversold_units': 0, 'anomaly_type': 'deadlock', 'timestamp': '1781594556.607525'}
最终建议: 你好。作为系统架构师，针对本次秒杀压测闭环中暴露出的 'deadlock' 及系统雪崩问题，我进行了深度的数据与日志剖析。

海南系统的核心症结在于：
**流量漏斗模型失效，大量并发请求穿透至数据库层，导致热点行锁竞争与连接池耗尽，最终引发死锁和系统雪崩。**

以下是详细的排查分析与调优建议：

### 一、 异常根因深度剖析

结合压测指标与系统日志，导致 `deadlock` 和 `TPS` 仅为 `80` 的根因可归结为以下四点：

1. **热点行锁竞争（核心瓶颈）**：`1500` 并发下，大量请求同时执行 `UPDATE inventory SET stock = stock - 1 WHERE sku_id = ?`。数据库对同一行（热点行）加排他锁（X锁），导致后续请求全部阻塞等待，行锁等待时间急剧增长。
2. **连接池饱和与雪崩效应**：由于行锁等待时间过长，大量数据库连接被占用且无法释放，连接池迅速被打满（`connection pool are saturated`），新请求无法获取连接，导致超时、重试，进而引发级联故障，被监控误报或衍生为真正死锁。
3. **事务范围过大（长事务）**：如果在扣减库存的事务中，还包含了创建订单、扣减余额、发送积分等耗时操作，会导致行锁持有时间成倍增加，极大加剧了锁竞争。
4. **Redis 前置拦截失效**：总请求 `11033`（平均 QPS 约 `1100`），但 `redis_qps` 仅为 `3578`（若为总调用次数，说明单请求多次调用；若为拦截失败，说明拦截率极低）。大量请求未被 Redis 拦截，直接穿透到了 DB 层。

### 二、 Redis 防超卖与原子扣减机制优化

必须将库存校验和扣减动作前置到 Redis，利用 Redis 的单线程特性挡住 95% 以上的无效请求。

#### 1. 原子扣减 Lua 脚本设计
严禁在应用层使用 `GET` 判断后再 `DECR`，必须使用 Lua 脚本保证“查库存-判资格-扣库存”的原子性。

```lua
-- Lua
-- 秒杀原子扣减 Lua 脚本
-- KEYS[1]: 库存 Key (如 sku:flash-sale-demo:stock)
-- KEYS[2]: 已购用户集合 Key (如 sku:flash-sale-demo:bought_users)
-- ARGV[1]: 扣减数量 (通常为 1)
-- ARGV[2]: 用户 ID

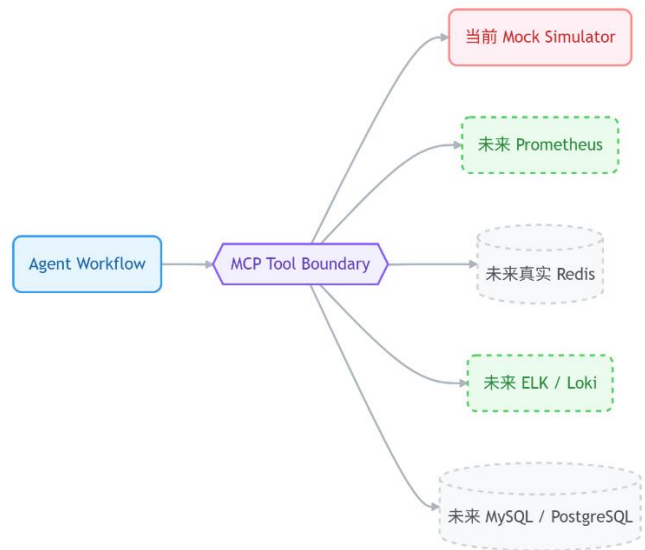
-- 1. 检查用户是否已购买 (防一人多买)
if redis.call('sismember', KEYS[2], ARGV[2]) == 1 then
 return -2 -- 返回 -2 表示重复购买
end
```

正常基线测试截图：

```
(cs599_env) E:\cs599-project>python -c "from src.target_system.flash_sale import FlashSaleSimulator; s=FlashSaleSimulator(seed=599); print(s.run(concurrency=500, inventory=800, duration_seconds=10))"
```

```
{'sku_id': 'sku:flash-sale-demo', 'concurrency': 500, 'inventory': 800, 'duration_seconds': 10, 'request_count': 682, 'success_orders': 669, 'failed_requests': 13, 'tps': 66.9, 'redis_qps': 151.53, 'remaining_inventory': 131, 'oversold_units': 0, 'anomaly_type': 'none', 'logs': ['2026-06-16 15:12:44 [INFO] flash sale round finished normally; inventory deduction is consistent'], 'timestamp': 1781593964.4417038}
```

得益于我们在项目初期规划的规格驱动与协议解耦设计，当前系统拥有极其强悍的横向演进空间。



虽然目前的 Target Layer 使用的是 Python 编写的本地 Mock 模拟器，但核心的通讯桥梁 MCP Server 已经标准化了所有的数据出入格式。这意味着在接下来的扩展中，只要保持 Agent 编排层代码不动，我们仅需重写 MCP Server 内部的具体实现逻辑，就可以通过 API Token 让系统无缝地去读取公司内部真实 K8s 集群中的 Prometheus 监控指标体系，或是直接拉取真实生产环境里的 ELK 日志大盘。整个替换过程属于完全透明的平滑迁移。

## 6.2 下一阶段计划

在未来的工程迭代中，系统计划向以下更深的智能化运维水准迈进：

**建立长效演练库：**将每一次压测产生的 JSON 态势数据和 Tuner 的优化建议持久化到向量数据库中。当下一次发生类似报错时，Tuner Agent 可以首先进行 RAG（检索增强生成）查询历史病历，避免重复诊断。

**报表自动化闭环：**引入 Report Agent 角色，在压测结束后，自动将晦涩的终端日志排版生成图文并茂的 Markdown 或 PDF 性能演练报告，自动通过企微/钉钉群推送给业务研发负责人。

**执行器与自愈闭环：**演进出具有物理修改权限的 Executor Agent。在获得人类主管的一键 Approval（审批确认）后，由 Agent 自动登录远程服务器，执行 Nginx 限流阈值的调整，或是动态扩容数据库连接池的配置，完成真正意义上的“故障闭环自愈”。

## 6.3 AI 能力演进路径

在企业级落地中，AI 能力的渗透不应是盲目的全盘接管，而是遵循着渐进式的演进路径。



第一阶段是规则兜底期，依靠正则表达式与 MockLLM 进行简单的异常拦截提示；

第二阶段是目前系统所处的云端辅助期，利用强大的千问大模型根据异常上下文进行深层关联分析并出具代码建议；

第三阶段是趋势洞察期，系统需要开始接入历史时序数据库，大模型不仅要看当前报错，还要结合过去一个月的流量波形进行风险前置预测；

最终的演进形态将是人机协同的自动驾驶期，大模型不再仅仅输出建议文本，而是直接输出包含 Terraform 或 K8s YAML 配置文件的自动化补丁，在经过极简的人工二次核验后直接下发上线。大模型的定位最终将蜕变为最硬核的架构重构执行引擎。

# 七、课程总结

## 7.1 个人收获

通过《企业级应用软件设计与开发》这门课程的学习，以及对 Chaos-Tuner Agent 项目在本地从零到一的完整构建，我对 AI 技术在传统软件工程中的改造力量有了极其震撼的实质性体验。

在此之前，我并未尝试甚至还未开始学习 agent 相关知识和项目实践。但这个项目让我深刻

地见识到，当利用 LangGraph 将模型的逻辑推理能力嵌套进一个有着严格状态约束和循环重试机制的状态机闭环中，并使用 MCP 协议为其装上触探真实物理环境的“手眼”之后，一个孤立的聊天接口竟然能够真正蜕变为一名能自动执行压测、秒级定位高并发数据库死锁，并直接写出极高质量架构调优代码的“数字资深架构师”。这种跨界融合的工程实践带给了我极大的启发。

## 7.2 工程思维转变

本项目促成了我最核心的一次工程思维跃升：视角的转变。

我真正开始从传统的“业务代码编写者”思维，蜕变为了“智能体编排者与规则制定者”思维。

在整个项目中，我花费了大量精力通过 SDD 文档去约束输入输出，利用 State 字典严防上下文污染，通过断网 Mock 机制去对抗模型调用的不可靠性，以及通过 LangSmith 去追求多步推理的白盒可观测性。我彻底明白了，真正的企业级 Agentic AI 是需要由严密的传统工程约束、健壮的安全边界隔离以及顶级的观测追踪体系共同羁绊住的超级算力引擎。

## 7.3 对课程的建议

这门课程站在了当今软件工程界技术变革的最前沿，对于开阔研究生的架构视野极具价值。关于未来的课程建设，我有以下建议：

课程虽然有抽签模式让部分同学参与到了实现，答辩互动的部分，但是大部分同学其实并没有切实的在课程过程中实践体会，我认为后续的课程老师可以准备一些较为基础简单的功能实现在课堂上让学生去实践，诱发大家对 agent 学习的兴趣，或者可以提供一个传统的系统，让学生们自发创新性地设计并开发智能改造，我认为相较于从零开始，在一个基本的框架上进行开发更能让同学们坚持下去。